

Coding Sample: Escaping Inequality in India (Selected Excerpts)

Rishav Roy

What follows are excerpts from the 138-page code appendix to my independent paper and writing sample, “Escaping Inequality in India: The Role of English-Medium Instruction.”

The next 47 pages include the following excerpts, each a part of the code chunks below it:

14-16: Scalable **import, cleaning, and merging** of multi-file census and geospatial data.

- Chunk 4: Import key data files

31-32: **QA and identifier-disambiguation checks** for district-level crosswalk.

- Chunk 6: Match districts: Construct district tracking dfs

50-52: **Testing for systematic missingness** using the Benjamini–Hochberg procedure and parallelized logistic regressions.

- Chunk 8: Participation model: Are NAs randomly distributed

55-57: **Survey-weighted probit estimation**, average marginal effects derivation, and setup for sample selection correction under a complex survey design.

- Chunk 9: Participation model: Estimation and IMR
- Chunk 10: Participation model: Derive AME

80-88: Reusable **record-linkage functions** using cascading fuzzy matches and unmatched/false-match diagnostics to harmonize district identifiers across years and data sources.

- Chunk 16: Match districts: Test joining methods
- Chunk 17: Match districts: Define joining functions

105-107: Reusable **IV specification and estimation pipeline** with clustered inference, plus contiguity-based spatial weights and setup for spatially lagged models.

- Chunk 24: Geospatial: Neighbor list construction
- Chunk 25: 2SLS: Controls and 2SLS formula
- Chunk 26: 2SLS: Baseline 2SLS models

111-112: **Robustness checks and diagnostics** for multicollinearity, spatial autocorrelation.

- Chunk 28: Multicollinearity check
- Chunk 29: Geospatial: Spatial autocorrelation tests

117-138: **Automated generation of publication-quality figures and tables** (including both summary statistics and regression outputs).

- Chunks 31-40: [Output for paper]

```

#### From the Census of India 2001
data_folder <- "Indian Census 2001"

# start.time <- Sys.time()
# 35 files will be read in. Preallocate list to store their cleaned data
files <- list.files(
  data_folder,
  pattern = "^PC01_C16_\\d{2}\\\\.xls$",
  full.names = TRUE
)
result_list <- vector("list", length(files))

for (i in seq_along(files)) {
  file_path <- files[i]
  temp <- read_excel(file_path, skip = 6, col_names = FALSE)

  # Explicitly set column names
  colnames(temp) <- c("table", "state_code", "district_code", "tehsil_code", "area_name",
    "mother_tongue_code", "mother_tongue", "spkr_tot", "m_spkr_tot",
    "f_spkr_tot", "spkr_urban", "m_spkr_urban", "f_spkr_urban",
    "spkr_rural", "m_spkr_rural", "f_spkr_rural")

  # Extract the state name:
  # Find the first row in the area_name column that begins with "State - "
  state_val <- temp$area_name[grepl("^State - ", temp$area_name)]
  if(length(state_val) == 0) {
    state <- NA_character_
  } else {
    state <- state_val[1] %>%
      # Remove the "State - " prefix

```

```

    str_remove("^State -\\s*") %>%
    # Remove four-digit suffix code
    str_remove("\\s*\\d{4}$") %>%
    # Remove leading/trailing spaces. Squish internal spaces
    str_squish() %>%
    # Convert to title case
    str_to_title()
  }

# Filter the dataframe for rows where both:
# 1. "mother_tongue_code" ends with "0"
# 2. "area_name" begins with "District - "
temp <- temp %>%
  filter(str_detect(as.character(mother_tongue_code), "0$"),
         str_detect(area_name, "^District - "))

# Create "district" and "state" columns
temp <- temp %>%
  mutate(
    # Create "district" by cleaning "area_name"
    district = area_name %>%
      str_remove("^District -\\s*") %>%
      str_remove("\\s*\\d{4}$") %>%
      str_squish() %>%
      # Remove all trailing non-letters
      str_replace("[^[:alpha:]]+$", ""),

    # Create "state" via extracted state name
    state = state
  )

# Clean "mother_tongue" column

```

```

temp <- temp %>%
  mutate(mother_tongue = mother_tongue %>%
    str_remove("^\\d{1,3}\\s+") %>% # Remove leading 1-3 digits and a space
    str_to_title()) # Title case

# Save cleaned df in list
result_list[[i]] <- temp
}

# Combine cleaned dfs
census01 <- bind_rows(result_list)

# end.time <- Sys.time()
# end.time - start.time
# For us, above order (clean files, then bind) was 1.48 secs faster than alternatives.

# Uncomment below to visually verify results
# census01 %>% select(-c(table, tehsil_code, area_name, mother_tongue_code)) %>%
  ↪ group_by(state) %>% distinct(district) %>% View()

# ---

#### District boundaries and regions ####
# From @meyersIndiaOfficialBoundaries2020 via @bhatiaMergingUpdatedDistrictlevel2020
data_folder <- "District Boundaries 2020"

district_bnds_20 <- st_read(
  file.path(data_folder, "district", "in_district.shp"),
  quiet = TRUE) %>%
  rename(district_20 = dtname,
    state_20 = stname) %>%

```

```

# View number of districts each year with the same name despite being in different states

# 1. Filter rows
# For each suffix, we flag rows where the district (for that suffix) appears with more
  ↳ than one distinct state. Then we keep rows where at least one of the suffix
  ↳ comparisons is TRUE.

# Create a list of logical vectors (one per suffix)
keep_list <- lapply(year_suffixes, function(sfx) {

  # Build the column names for this suffix
  state_col <- paste0("state_", sfx)
  district_col <- paste0("district_", sfx)

  # Group the data by district for this suffix and count distinct states
  temp <- district_tracker %>%
    filter(
      !is.na(.data[[district_col]]),
      !is.na(.data[[state_col]])
    ) %>%
    group_by(district = .data[[district_col]]) %>%
    summarise(
      n_states = n_distinct(.data[[state_col]], na.rm = TRUE),
      .groups = "drop"
    )

  # Districts that occur with more than one unique state
  duplicate_districts <- temp %>%
    filter(n_states > 1) %>%
    pull(district)

  # Return a logical vector: TRUE if the district value in the row is in
  ↳ duplicate_districts

```

```

    district_tracker[[district_col]] %in% duplicate_districts
  })

# Combine the logical vectors by taking the rowwise OR (i.e., TRUE if condition holds for
↪ any suffix)
keep_indicator <- Reduce(`|`, keep_list)

# Filter the original data
same_name_districts <- district_tracker[keep_indicator, ]


# 2. Summarize by district for each suffix
# For each suffix, group the filtered data by the district column and count:
# a. the number of rows (n_rows) sharing that district value, and
# b. the number of distinct state values (n_states) in that group.
# Then keep only groups that have more than one unique state.
temp <- lapply(year_suffixes, function(sfx) {
  state_col <- paste0("state_", sfx)
  district_col <- paste0("district_", sfx)
  same_name_districts %>%
    group_by(district_name = .data[[district_col]]) %>%
    summarise(n_rows = n(),
              n_states = n_distinct(.data[[state_col]]),
              .groups = "drop") %>%
    filter(n_states > 1) %>%
    mutate(year_suffix = sfx)
})

# Combine summaries from all suffixes into one data frame
temp <- bind_rows(temp)

# Make it easy to get summary stats
n_same_name_districts <- temp %>%
  group_by(year_suffix) %>%
  summarize(n = n())

# Between min(n_same_name_districts$n) = 3 and max(n_same_name_districts$n) = 5 districts
↪ with the same name in different states in each year of interest

```

```

)

#### Logistic per missing variable ####

check_missing_logit_parallel <- function(df, miss_vars, covars, method_p = "BH") {
  # RHS of regression
  rhs <- paste(covars, collapse = " + ")

  # To fit the binary outcome is.na(m) for each missing var m
  fit_one <- function(m) {
    f <- as.formula(paste0("is.na(", m, ") ~ ", rhs))
    fit <- glm(f, data = df, family = binomial)
    pseudoR2 <- 1 - fit$deviance / fit$null.deviance # Note: No missingness indicator was
    ↪ constant
    gl <- glance(fit) # Currently unused; retained for future diagnostics

    # Return tidy df of coefficients and attach missing var's name
    broom::tidy(fit) %>% # Namespace made explicit for cluster workers
      mutate(
        missing_var = m,
        pseudoR2 = pseudoR2,
        nobs = stats::nobs(fit)
      )
  }

  # Parallelization
  # Cluster with parLapply() if on Windows
  out_list <- if (.Platform$OS.type == "windows") {
    cl <- parallel::makeCluster(max(1, parallel::detectCores() - 1))
    on.exit(parallel::stopCluster(cl), add = TRUE) # Shut down cluster even if errors
    ↪ occur
  }

```

```

parallel::clusterEvalQ(cl, {
  library(broom)
  library(dplyr)
})
parallel::clusterExport(
  cl,
  varlist = c("df", "rhs", "fit_one"),
  envir = environment()
)
parallel::parLapply(cl, miss_vars, fit_one)
} else { # Fork with mclapply()
  parallel::mclapply(miss_vars, fit_one, mc.cores = max(1, parallel::detectCores() -
↪ 1))
} # Returns list of tidy dfs, one per missing var

out <- bind_rows(out_list) # Stack the dfs
out <- out %>%
  group_by(missing_var) %>%
  mutate(
    p_adj = { # Adjust within each missingness model
      adj <- rep(NA_real_, dplyr::n())
      idx <- term != "(Intercept)" # Exclude intercept
      adj[idx] <- p.adjust(p.value[idx], method = method_p)
      adj
    }
  ) %>%
  ungroup()
out
}

covars <- c("SECTOR", "SEX", "AGE", "HH_SIZE", "RELIGION", "SOCIAL_GROUP", "state_0708")

# Run on missing variables defined for all children

```



```

logit_missing_results_all <- check_missing_logit_parallel(selection_df, miss_vars_all,
  ↪ covars, method_p = "BH") # "BH" = Benjamini-Hochberg control on false discovery rate.
  ↪

# "BY" lets independence or positive dependence assumptions weaken to arbitrary
  ↪ dependence, but has weaker power (i.e., more p-vals above 0.05).

sel_logit_NA_all <- logit_missing_results_all %>%
  group_by(missing_var) %>% # Use name from fit_one()
  summarise(
    n_sig = sum(p_adj < 0.05 & term != "(Intercept)",
    pseudoR2 = max(pseudoR2, na.rm = TRUE) # Model-level metric
  ) %>%
  arrange(desc(pseudoR2))
# So n_sig = # of predictors which survive FDR at 0.05 significance (i.e., # of
  ↪ covariates associated with missingness)

# Predictors of missingness for missing vars in probit (dmean_num_ENROLLMENT_COST and
  ↪ DIST_FROM_NEAREST_PRIMARY_CLASS)
sel_predlogit_NA_all <- logit_missing_results_all %>%
  filter(
    missing_var %in% miss_vars,
    term != "(Intercept)",
    p_adj < 0.05) %>%
  mutate(predictor = term) %>%
  select(missing_var, predictor, estimate, p_adj) %>%
  arrange(missing_var, p_adj, desc(abs(estimate)))

# Plot results: variables with most non-random missingness
plt_sel_logit_NA_all <- logit_missing_results_all %>%
  group_by(missing_var) %>%
  summarise(pseudoR2 = max(pseudoR2, na.rm = TRUE)) %>%
  ggplot(aes(reorder(missing_var, pseudoR2), pseudoR2)) +

```

```

# "'remove' and 'certainty' options give the same result"
# "but 'certainty' is intended for situations where there is only one PSU in the
  ↳ population stratum, which is sampled with certainty (also called 'self-representing'
  ↳ PSUs or strata). With 'certainty' no warning is generated for strata with only one
  ↳ PSU. Ordinarily, svydesign will detect certainty PSUs, making this option
  ↳ unnecessary."

# Will thus use "average" for handling single-PSU strata (e.g., "Jhelam Valley") when
  ↳ estimating variance
options(survey.lonely.psu = "average")

# Create weighted survey design model
selection_design <- svydesign(

  ids = ~FSU_SL_NO + HHID,

  # `?svydesign`: "For multistage sampling the id argument should specify a formula with
    ↳ the cluster identifiers at each stage."

  # Sampling occurred in two stages: the first-stage units (FSU) were rural
    ↳ villages/urban blocks (FSU_SL_NO), and the ultimate stage units (USUs) were
    ↳ households (HHID)

  strata = ~interaction(
    as.character(STATE),
    as.character(STRATUM),
    as.character(SUB_STRATUM_NO),
    drop = TRUE # Nothing dropped yet
  ),

  # Sampling stratified into state, stratum (district), and substratum (rural/urban, with
    ↳ multiple urban substrata if population dense enough)

  weights = ~weight,
  data = selection_df,

```

```

    nest = TRUE # Clusters (PSUs) are nested within strata (states*strata*substrata)
)

# Estimate weighted model
model_probit_selection <- svyglm(
  f_probit,
  design = selection_design,
  family = binomial(link = "probit")
) # Rows with an NA for a probit-relevant variable dropped!

# Derive IMR
# For now, calculate IMR for all rows. Don't drop rows with NAs in unrelated columns:

# Rows with no NA under probit-relevant variable
needed_vars <- all.vars(f_probit) # = c(probit_vars, "enrolled")

# Take only complete cases of variables in f_probit
comp_cases <- complete.cases(selection_df[, needed_vars, drop = FALSE])

# Predict only on complete rows; NA for rest
lp <- rep(NA_real_, nrow(selection_df)) # full-length placeholders
lp[comp_cases] <- predict(model_probit_selection,
                          newdata = selection_df[comp_cases, , drop=FALSE],
                          type = "link") # to return linear predictors

# Build IMR
phi_lp <- dnorm(lp)
Phi_lp <- pnorm(lp)

```

```

# Remaining NAs could be from dividing by 0.
# Ensure against this by keeping Phi_lp \in (0,1) strictly
Phi_lp <- pmin(pmax(Phi_lp, .Machine$double.eps), 1 - .Machine$double.eps)
# .Machine$double.eps = 2.220446e-16 on my device

selection_df <- selection_df %>%

  mutate(
    IMR = case_when(
      enrolled == "Yes" ~ phi_lp / Phi_lp,
      enrolled == "No" ~ -phi_lp / (1 - Phi_lp),
      TRUE ~ NA_real_ # NA if missing predictor
    )
  )

```

Chunk 10: Participation model: Derive AME

```

# Use automatic differentiation (AD) for SE delta method gradients
autodiff(autodiff = TRUE, install = FALSE) # install=TRUE if self-managing Python install
options(marginalEffects_parallel = FALSE) # Make sure it's off
sel_data <- selection_df[comp_cases, , drop = FALSE]

num_samp <- 200
set.seed(999)
final_draft <- TRUE # Set TRUE when rendering final draft
newdata_ame <- if (final_draft) sel_data else sel_data %>% slice_sample(n = num_samp)

# start.time <- Sys.time()
mfx_all <- avg_slopes(
  model_probit_selection,
  newdata = newdata_ame,
  wts = "weight",

```

```

# # More manual tests
# stringdist("East Godavari", "Godavari East", method = "qgram")
# # 0
# stringdist("East Godavari", "Godavari East", method = "jaccard")
# # 0

evaluate_distances <- function(pairs,
                               methods, thresholds,
                               col1 = "str1", col2 = "str2") {
  if(length(methods) != length(thresholds)) {
    stop("\"methods\" and \"thresholds\" must have the same length.")
  }
  # Ensure chr type
  pairs <- pairs %>%
    mutate(
      !!col1 := as.character(.data[[col1]]),
      !!col2 := as.character(.data[[col2]])
    )

  # Compute distances for each method
  out <- bind_rows(
    lapply(seq_along(methods), function(i) {
      m <- methods[i]
      th <- thresholds[i]

      pairs %>%
        transmute(
          str1 = .data[[col1]],
          str2 = .data[[col2]],
          method = m,
          distance = stringdist(str1, str2, method = m),

```

```

    threshold = th,
    match = distance <= th
  )
})
)

# Keep a consistent ordering
out %>% arrange(str1, str2, method)
}

# Some sample pairs
pairs <- tribble(
  ~str1, ~str2,
  "Baleshwar", "Balasore",
  "Jammu & Kashmir", "Jammu and Kashmir",
  "East Godavari", "Godavari East",
  "Sikim", "Sikkim",
  "Mumbai", "Mumbai",
  "24-Parganas ( North )", "North Twenty Four Parganas",
  "North Twenty Four Pargan*", "North Twenty Four Parganas",
  "Sahibzada Ajit Singh Nag*", "Sahibzada Ajit Singh Nagar",
  "Sri Potti Sriramulu Nell*", "Sri Potti Sriramulu Nellore"
)

#### Helper vectors ####
methods <- c("soundex", "qgram", "jw", "dl", "osa")
thresholds <- c(0, 0, 0.15, 2, 1)

# soundex=0 --> Phonetic variants in anglicization allowed (if coarsely measured)
# qgram = 0 --> Some rearrangements of characters allowed
# jw<=0.15 --> Respellings + vowel swaps with 0.85 similarity allowed
# dl<=2 --> <=2 insertions + deletions + substitutions + transpositions allowed

```

```

# osa<=1 --> 1 typo allowed
# Template for testing these vs. other methods/thresholds can be found at start of chunk

# Evaluate distances and test joining methods
# colSums(is.na(joined_df))
# # 53 in 01, 112 in 07-08, 89 in 17-18
# sum(apply(joined_df, 1, anyNA))
# # 166/734
# evaluate_distances(pairs, methods, thresholds)

```

Chunk 17: Match districts: Define joining functions

```

# A. Loop over the methods*thresholds
# B. For each methods*thresholds pair, fuzzy full join all still-unmatched rows
# C. Return joined, unmatched_df1, unmatched_df2
fuzzy_join_sequence <- function(df1, df2,
                                dist1, state1,
                                dist2, state2,
                                methods, thresholds, mode = "full") {
  # Attach row IDs to track unmatched
  df1_id <- df1 %>% mutate(.id1 = row_number())
  df2_id <- df2 %>% mutate(.id2 = row_number())

  df1_curr <- df1_id
  df2_curr <- df2_id
  matched_all <- tibble()

  for(i in seq_along(methods)) {
    full_j <- stringdist_join(
      df1_curr, df2_curr,

```

```

    by = setNames(c(dist2, state2), c(dist1, state1)),
    mode = mode,
    method = methods[i],
    max_dist = thresholds[i],
    distance_col = "dist"
  )

  # Keep only pairs (i.e., those with both sides present) and rank them
  matched_i <- full_j %>%
    filter(!is.na(.id1) & !is.na(.id2)) %>%
    group_by(.id1) %>%
    slice_min(order_by = dist, n = 1, with_ties = FALSE) %>%
    ungroup() %>%
    group_by(.id2) %>%
    slice_min(order_by = dist, n = 1, with_ties = FALSE) %>%
    ungroup()
  matched_all <- bind_rows(matched_all, matched_i)

  # Remove them from the to-do sets
  df1_curr <- df1_curr %>% filter(!(.id1 %in% matched_i$.id1))
  df2_curr <- df2_curr %>% filter(!(.id2 %in% matched_i$.id2))
}

list(
  joined = matched_all,
  unmatched_df1 = df1_curr %>% select(-.id1),
  unmatched_df2 = df2_curr %>% select(-.id2)
)
}

# Default is full join so that this can be used for both matching and diagnostics

```



```

# For each df name in input vector:
# A. Extract its "01"/"05"/... suffix
# B. Find the suffix's district_/state_ columns in joined_df.
# C. Run the fuzzy join sequence on each column. (Recall justification of fuzzy joining
  ↳ states in Chunk 6: Match districts: Construct district tracking dfs.)
# D. Repeat with the predecessor suffix.
merge_dfs_into_tracker <- function(
  dfs,
  tracker,
  years_of_interest,
  methods, thresholds,
  flag = FALSE
) {
  # 1. Initialize both joined_df and (if flag=TRUE) flagged_df
  joined_df <- tracker
  if (flag) {
    flagged_df <- tracker
    # Prepare placeholder vector for each df-name
    matched_lists <- lapply(names(dfs), function(x) rep(NA_character_, nrow(tracker)))
    names(matched_lists) <- names(dfs)
  }

  year_suffixes <- substr(as.character(years_of_interest), 3, 4)
  unmatched_list <- vector("list", length(dfs))
  names(unmatched_list) <- names(dfs)

  # 2. Loop over each auxiliary data-frame name
  for (d_name in names(dfs)) {
    df_d <- dfs[[d_name]]
    suffix <- stringr::str_extract(d_name, "\\d{2}$")
    stopifnot(!is.na(suffix))
  }
}

```

```

# Find exactly one district_XX / state_XX in df_d
dist_d <- grep("^district_", names(df_d), value=TRUE) %>%
  grep("code", ., invert=TRUE, value=TRUE)
state_d <- grep("^state_", names(df_d), value=TRUE) %>%
  grep("code", ., invert=TRUE, value=TRUE)
stopifnot(length(dist_d)==1, length(state_d)==1)

df_unmatched <- df_d
if (flag) mv <- matched_lists[[d_name]]

# suffix_chain sorted by absolute distance from said suffix
year_nums <- as.integer(year_suffixes)
suffix_num <- as.integer(suffix)
order_idx <- order(abs(year_nums - suffix_num))
suffix_chain <- year_suffixes[order_idx]

# Work through suffixes, closest to furthest
for (suf in suffix_chain) {
  dist_cols_j <- grep(paste0("^district_", suf, "(?:$|_)"),
    names(joined_df), value=TRUE)
  state_cols_j <- grep(paste0("^state_", suf, "(?:$|_)"),
    names(joined_df), value=TRUE)

  for (dist_col_j in dist_cols_j) {
    ending <- sub(paste0("^district_", suf), "", dist_col_j)
    state_col_j <- paste0("state_", suf, ending)
    if (!state_col_j %in% state_cols_j) next
  }

  # 2a. Multi-threshold fuzzy join
  res <- fuzzy_join_sequence(
    df_unmatched, joined_df,
    dist1 = dist_d,

```

```

state1 = state_d,
dist2 = dist_col_j,
state2 = state_col_j,
methods = methods,
thresholds = thresholds,
mode = "full"
)

# 2b. Pull other cols into joined_df
other_cols <- setdiff(names(df_d), c(dist_d, state_d))
if (nrow(res$joined)>0 && length(other_cols)>0) {
  extra <- res$joined %>%
    select(.id2, all_of(intersect(other_cols, names(res$joined)))) %>%
    arrange(dist, .id2) %>%
    group_by(.id2) %>%
    slice_min(order_by = dist, n = 1, with_ties = FALSE) %>%
    ungroup()

  # Merge into joined_df
  joined_df <- joined_df %>%
    mutate(.row = row_number()) %>%
    left_join(extra, by = c(".row"=".id2")) %>%
    select(-.row)

  # If flag, merge into flagged_df
  if (flag) {
    flagged_df <- flagged_df %>%
      mutate(.row = row_number()) %>%
      left_join(extra, by = c(".row"=".id2")) %>%
      select(-.row)
  }
}

```

```

# 2c. If flag=TRUE, record which district got matched this iteration
if (flag && nrow(res$joined)>0) {
  col_match <- grep(
    paste0("^", dist_d, "(?:$|\\.x$)"),
    names(res$joined), value=TRUE
  )[1]
  info <- res$joined[, c(".id2", col_match), drop=FALSE]
  colnames(info)[2] <- "match_dist"
  info <- info %>%
    group_by(.id2) %>%
    slice_head(n = 1) %>%
    ungroup()

  idx <- info$.id2
  new_vals <- info$match_dist
  blank <- is.na(mv[idx])
  mv[idx[blank]] <- new_vals[blank]
}

# 2d. Update the df of unmatched
df_unmatched <- res$unmatched_df1
if (nrow(df_unmatched)==0) break
}

if (nrow(df_unmatched)==0) break
}

# 2e. Once done, append flag-column to flagged_df
if (flag) {
  flagged_df[[d_name]] <- mv
}
unmatched_list[[d_name]] <- df_unmatched

```

```

    if (nrow(df_unmatched)>0) {
      warning(sprintf("Some rows in %s were never matched", d_name))
    }
  }

# 3. Finalize flagged_df if flag=TRUE
if (flag) {
  max_suf <- year_suffixes[which.max(as.integer(year_suffixes))]
  keep_cols <- names(flagged_df)[
    (grepl("^district_", names(flagged_df)) & !grepl("code", names(flagged_df))) |
    names(flagged_df) %in% names(dfs) |
    names(flagged_df) == paste0("state_", max_suf)
  ]
  flagged_df <- flagged_df[, keep_cols, drop=FALSE]
}

# 4. Return!!!
if (flag) {
  list(
    joined_df = joined_df,
    unmatched = unmatched_list,
    flagged_df = flagged_df
  )
} else {
  list(
    joined_df = joined_df,
    unmatched = unmatched_list
  )
}
}

```

```

# rook_neighbors <- joined_geom %>% st_rook()
# rook_neighbors %>% lengths() %>% mean()
# # 4.780165 average neighbors per district
# queen_neighbors <- joined_geom %>% st_queen()
# queen_neighbors %>% lengths() %>% mean()
# # 4.783471
# # Average neighbor counts are nearly equivalent

# # Test for fastest contiguity measure
# start.time <- Sys.time()
nb_2020 <- joined_df %>%
  poly2nb(queen = FALSE)
# end.time <- Sys.time()
# end.time - start.time
# # With queen = TRUE: 32.64976 secs
# # With queen = FALSE: 31.15051 secs
# # Rook contiguity slightly faster

# Binary adjacency matrix. Rook-contiguity neighbor list used for speed, conceptual
#  → interpretability
W_2020 <- nb2mat(nb_2020, style = "B", zero.policy = TRUE)
# W_2020[i,j] == 1 if districts i and j share a border, 0 else

```

Chunk 25: 2SLS: Controls and 2SLS formula

```

#### Control vectors ####
controls_needlag <- c("consumption_0708", "gini_cons_0708")
controls_nolag <- c(
  "pct_urban", "avg_hh_size", "dependency_ratio",
  "pct_fem_head",
  "pct_hindu", "pct_muslim",

```

```

    "pct_st", "pct_sc", "pct_obc",
    "pct_small_land", "pct_medium_land", "pct_large_land",
    "pct_head_lit_to_primary",
    "pct_head_secondary_plus")

# Extract spatial lags
controls_lagged <- paste0("W_", controls_needlag)

#### IV Formula ####
make_iv_formula <- function(dep, endog, exog, inst) {
  as.formula(paste0(
    dep, " ~ ",
    paste(c(endog, exog), collapse = " + "),
    " | ",
    paste(c(inst, exog), collapse = " + ")
  ))
}

```

Chunk 26: 2SLS: Baseline 2SLS models

```

# Formula syntax: outcome ~ endogenous + controls | instrument + controls
model_consumption_iv <- ivreg(
  make_iv_formula(
    dep = "consumption_pct_change",
    endog = "EMIE",
    exog = c(controls_needlag, controls_nolag),
    inst = "wavg_ling_degrees"
  ),
  data = joined_df
)

```

```

# First-stage regression
first_stage_consumption <- lm(
  reformulate(
    c("wavg_ling_degrees", controls_needlag, controls_nolag),
    response = "EMIE"
  ),
  data = joined_df # Complete cases already enforced, so ivreg() and lm() will use the
  ↪ same estimation sample
)

# Cluster SEs by state
vcov_first_stage_consumption <- vcovCL(first_stage_consumption, cluster = ~ state_20)
vcov_model_consumption_iv <- vcovCL(model_consumption_iv, cluster = ~ state_20)

# Repeat for Gini model
model_gini_iv <- ivreg(
  make_iv_formula(
    dep = "gini_change",
    endog = "EMIE",
    exog = c(controls_needlag, controls_nolag),
    inst = "wavg_ling_degrees"
  ),
  data = joined_df
)

vcov_model_gini_iv <- vcovCL(model_gini_iv, cluster = ~ state_20)

```



```

    comma(round(.x, 0)),
    # Otherwise, keep two decimals
    sprintf("%.2f", .x)
  )
)
)# %>% mutate(across(Min:SD, ~ signif(.x, 3))) # For 3 significant digits

```

Chunk 28: Multicollinearity check

```

X <- c("EMIE", controls_needlag, controls_nolag) %>%
  reformulate() %>%
  model.matrix(data = joined_df)

# # Number of independent columns:
# qr(X)$rank
# # Num. total columns:
# ncol(X)
# # Recall: If qr(X)$rank < ncol(X), exact collinearity.

# Heuristic: kappa>30 implies severe multicollinearity
X_no_intercept <- X[, colnames(X) != "(Intercept)", drop = FALSE]
keep <- apply(X_no_intercept, 2, sd, na.rm = TRUE) > 0
X_scaled <- scale(X_no_intercept[, keep, drop = FALSE])
kappa_val <- kappa(X_scaled, exact = TRUE)

# # Check for linear dependence
# X_df <- as.data.frame(X_scaled)
# alias(lm(
#   consumption_pct_change ~ .,
#   data = cbind(consumption_pct_change = joined_df$consumption_pct_change, X_df)
# ))

```

```

# # Heuristic: VIF > 10 implies problematic collinearity
# vif(lm(
#   reformulate(
#     c("EMIE", controls_needlag, controls_nolag),
#     response = "consumption_pct_change"
#   ),
#   data = joined_df
# ))
#
# vif(model_consumption_iv)
# vif(model_gini_iv)

```

Chunk 29: Geospatial: Spatial autocorrelation tests

```

# Build row-standardized spatial weights (listw) object
# Recall: Number+order of rows in 2SLS models = number+order used for spatial weights
listw_2020 <- nb2listw(nb_2020, style = "W", zero.policy = TRUE) # TRUE for, say, islands

#Extract residuals from IV models
resid_cons <- residuals(model_consumption_iv)
resid_gini <- residuals(model_gini_iv)
resid_fscons <- residuals(first_stage_consumption)

# Global Moran's I on residuals
m_cons_resid <- moran.test(resid_cons, listw_2020, zero.policy = TRUE)
m_gini_resid <- moran.test(resid_gini, listw_2020, zero.policy = TRUE)
m_fscons_resid <- moran.test(resid_fscons, listw_2020, zero.policy = TRUE)

# # p-vals via Monte Carlo (moran.test() assumes asymptotic normality)
# set.seed(999)

```

Chunks 31–40: [Output for paper]

fig-ilo-trends (Figure 1)

```
# fig.pos='p': Placed on page with only floats (figs, tables, etc.)
files <- c(
  "Average Monthly Real Earnings Over Time - Total.png",
  "LFPR WPR and Unemployment for All Over Time.png",
  "Unemployment Rate By General Education.png"
)
paths <- file.path("580 Paper Images", files)

width_px <- 1300 # Reduce further if needed
imgs <- lapply(paths, function(p) image_read(p) |> image_scale(paste0(width_px)))

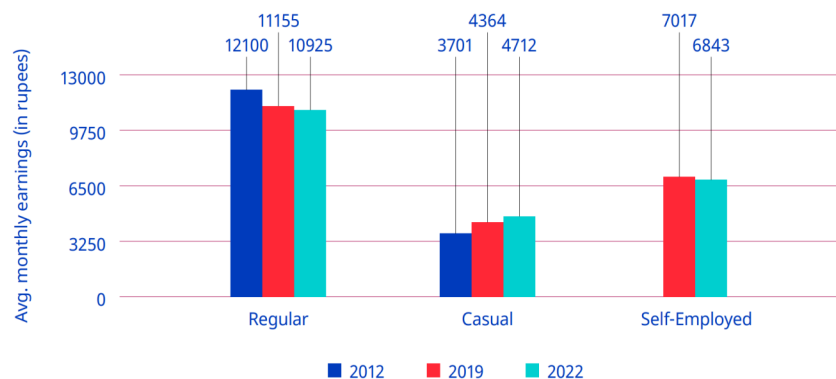
collage <- image_append(image_join(imgs), stack = TRUE)

out_path <- "ILO-fig.png"
image_write(collage, path = out_path)
knitr::include_graphics(out_path)
```

tab-probit-summary-numeric (Table 1)

```
kbl(
  sel_quant_stats %>%
    select(Variable=label, Min, `1Q`, Med, `3Q`, Max, Mean, SD, N),
  format = "latex",
  booktabs=TRUE,
  longtable=TRUE,
  label = "tab-probit-summary-numeric",
  caption="Summary Statistics for Enrollment Participation Model (Numeric Variables)",
  escape=FALSE
) %>%
```

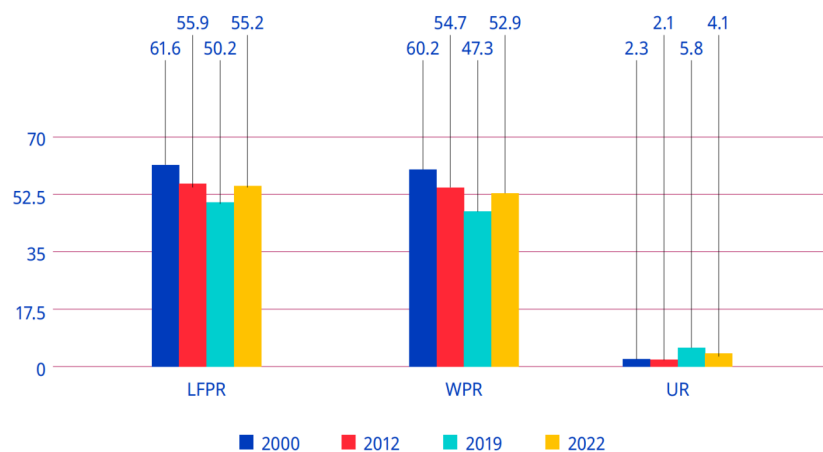
► Figure 2.8. Average monthly earnings of regular, casual and self-employed workers, 2012, 2019 and 2022 (rupees, at 2012 prices)



Note: Casual wages also included wages in public works, such as the Mahatma Gandhi National Rural Employment Guarantee. Wages were adjusted using the consumer price index-R and consumer price index-U by taking 2012 as the base year.

Source: Computed from various years of the Employment and Unemployment Survey data and the Periodic Labour Force Survey unit-level data.

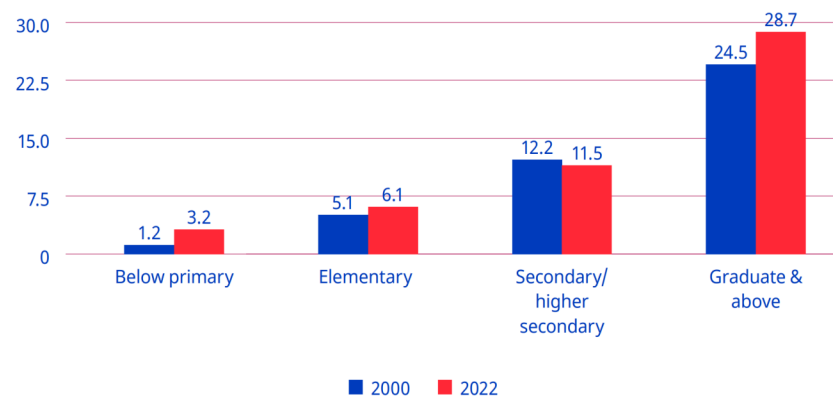
► Figure 2.1. Labour force participation rate, worker population ratio and unemployment rate (UPSS) among persons aged 15+ (rural and urban combined), 2000, 2012, 2019 and 2022 (%)



Note: LFPR=labour force participation rate; WPR=worker population ratio; UR=unemployment rate.

Source: Computed from various years of the Employment and Unemployment Survey data and the Periodic Labour Force Survey unit-level data.

► Figure 5.14. Unemployment rate for youths, by level of general education (UPSS), 2000 and 2022 (%)



Source: Computed from Employment and Unemployment Survey data for 2000 and Periodic Labour Force Survey data for 2022.

Figure 1: Trends in earnings, labor-force participation, and unemployment (ILO, 2024).

```

kable_styling(
  latex_options = c("hold_position", "repeat_header", "striped"),
  position = "center"
) %>%
# # Group district-level aggregate rows into single block
pack_rows("District-level aggregates:",
  which(sel_quant_stats$var=="dmean_num_IS_EDU_FREE"),
  nrow(sel_quant_stats)) %>%
footnote(
  general = paste(
    "`Min.' = minimum;",
    "`1Q' = first quartile;",
    "`Med.' = median;",
    "`3Q' = third quartile;",
    "`Max.' = maximum;",
    "`Mean' = arithmetic mean;",
    "`SD' = standard deviation;",
    "`N' = number of observations",
    collapse = " "
  ),
  escape = FALSE,
  threeparttable = TRUE,
  footnote_as_chunk = TRUE
) %>%
landscape()

```

Table 1: Summary Statistics for Enrollment Participation Model
(Numeric Variables)

Variable	Min	1Q	Med	3Q	Max	Mean	SD	N
Age	5.00	8.00	12.00	16.00	19.00	11.89	4.22	127246
Household size	1.00	4.00	5.00	7.00	30.00	5.91	2.40	127246
Enrollment cost (Rs.)	0.00	2,318.75	4,900.00	8,750.00	338,300.00	7,049.28	9,283.86	7184
District-level aggregates:								
Educ. free available? (Yes = 1)	0.00	0.48	0.69	0.85	1.00	0.65	0.23	127246
Tuition waived?	0.00	0.00	0.01	0.02	0.48	0.02	0.04	127246
Scholarship/Stipend?	0.00	0.01	0.05	0.20	0.90	0.13	0.17	127246
Textbooks received?	0.00	0.37	0.52	0.66	0.97	0.51	0.20	127246
Stationery received?	0.00	0.01	0.04	0.11	0.95	0.08	0.12	127246
Mid-day meal or more received?	0.02	0.33	0.42	0.55	0.90	0.43	0.17	127246

Note: ‘Min.’ = minimum; ‘1Q’ = first quartile; ‘Med.’ = median; ‘3Q’ = third quartile; ‘Max.’ = maximum; ‘Mean’ = arithmetic mean; ‘SD’ = standard deviation; ‘N’ = number of observations

tab-probit-summary-categorical (Table 2)

```
kbl(
  sel_cat_stats %>%
    select(Variable=label, Values, Mode, `"% Mode"`, `Least Freq.`, `"% Least Freq."`, N),
  format = "latex",
  col.names = c("Variable", "Values", "Mode", "Pct. Mode", "Least Freq.", "Pct. Least
    ↪ Freq.", "N"),
  booktabs=TRUE,
  longtable=TRUE,
  label = "tab-probit-summary-categorical",
  caption="Summary Statistics for Enrollment Participation Model (Categorical
    ↪ Variables)",
  escape=FALSE
) %>%
kable_styling(
  latex_options = c("hold_position", "repeat_header", "striped"),
  position = "center"
) %>%
footnote(
  general = paste(
    "`Values' = all possible values;",
    "`Mode' = most frequent value;",
    "`Pct. Mode' = Pct. of observations which take on the modal value;",
    "`Least Freq.' = least frequent value;",
    "`Pct. Least Freq.' = Pct. of observations which take on the least freq. value;",
    "`N' = number of observations.",
    collapse = " "
  ),
  escape = FALSE,
  threeparttable = TRUE,
  footnote_as_chunk = TRUE
) %>%
```

```
landscape() %>%  
# Set Variables column narrow, Values a bit wider, Mode smaller, etc.  
column_spec(1, width = "4cm") %>% # Variable  
column_spec(2, width = "6cm") %>% # Values  
column_spec(4, width = "2cm") %>% # Pct. Mode  
column_spec(6, width = "2cm") # Pct. Least Freq.
```


Table 2: Summary Statistics for Enrollment Participation Model
(Categorical Variables)

Variable	Values	Mode	Pct. Mode	Least Freq.	Pct. Least Freq.	N
Sex	Male, Female	Female	53.4	Male	46.6	127246
Religion	Hindu, Muslim, Christian, Sikh, Jain, Buddhist, Zoroastrian, Other	Hindu	75.2	Other	0.0	127246
Social group	Other, Scheduled Tribe, Scheduled Caste, Other Backward Class	Other Backward Class	39.5	Scheduled Tribe	14.2	127246
Urban	Rural, Urban	Rural	67.5	Urban	32.5	127246
Distance of nearest primary class	d<1km, 1km <= d <2kms, 2kms<= d <3kms, 3kms <= d <5kms, d>=5kms	d<1km	91.7	3kms <= d <5kms	0.1	126934
Father's education	Illiterate, Literate, no school, Literate, school < primary, Primary, Upper primary, Secondary, Higher secondary, Postsecondary+	Illiterate	55.3	Literate, no school	1.4	127179

Note: 'Values' = all possible values; 'Mode' = most frequent value; 'Pct. Mode' = Pct. of observations which take on the modal value; 'Least Freq.' = least frequent value; 'Pct. Least Freq.' = Pct. of observations which take on the least freq. value; 'N' = number of observations.

tab-probit-ame (Table 3)

```
# Print with kableExtra. Centered w/ booktabs style
kbl(
  reg_tbl_selection,
  format = "latex",
  booktabs = TRUE,
  longtable = TRUE,
  linesep = "",
  label = "tab-probit-ame",
  caption = "Average Marginal Effects for Enrollment Probit",
  align = c("l", "r", "r")
) %>%
kable_styling(
  latex_options = c("hold_position", "repeat_header", "striped"), # hold_position =
  ↪ Keep near chunk's position
  position = "center"
) %>%
add_header_above(c(" " = 1, "Enrolled in School (1 = yes)" = 2)) %>% # Dep. var. label
  ↪ above numeric vars
footnote(
  general = "Data from the 64th round of the NSS, ``Participation and Expenditure in
  ↪ Education'' in 2007-08. All standard errors are design-based, with PSUs clustered
  ↪ and nested within strata.",
  threeparttable = TRUE,
  escape = FALSE
)
```

Table 3: Average Marginal Effects for Enrollment Probit

Term	Enrolled in School (1 = yes)	
	Estimate	Std. Error
Age (years)	-0.029***	(0.000)
Female (ref: Male)	0.044***	(0.003)
Household size	-0.003***	(0.001)
Religion: Muslim (ref: Hindu)	-0.124***	(0.007)
Religion: Christian	-0.039**	(0.014)
Religion: Sikh	0.034	(0.026)
Religion: Jain	0.005	(0.012)
Religion: Buddhist	0.050*	(0.024)
Religion: Zoroastrian	0.105***	(0.032)
Religion: Other	0.254***	(0.002)
Social group: Scheduled Tribe (ref: Other)	-0.103***	(0.008)
Social group: Scheduled Caste	-0.076***	(0.006)
Social group: Other Backward Class	-0.030***	(0.005)
Urban (ref: Rural)	-0.006	(0.005)
Distance 1–2km (ref: <1km)	-0.068**	(0.025)
Distance 2–3km	-0.001	(0.010)
Distance 3–5km	0.007	(0.068)
Distance > 5km	0.043	(0.035)
Father’s educ.: Literate, no school (ref: Illiterate)	0.089***	(0.017)
Father’s educ.: Literate, school < primary	0.125***	(0.006)
Father’s educ.: Primary	0.141***	(0.005)
Father’s educ.: Upper primary	0.194***	(0.006)
Father’s educ.: Secondary	0.213***	(0.007)
Father’s educ.: Higher secondary	0.248***	(0.008)
Father’s educ.: Postsecondary+	0.239***	(0.011)
Educ. free available (ref: No)	-0.086***	(0.015)
Tuition waiver received	-0.060	(0.057)
Scholarship/Stipend received	0.022	(0.012)

Table 3: Average Marginal Effects for Enrollment Probit (*continued*)

Term	Estimate	Std. Error
Textbook(s) received	0.074***	(0.017)
Stationery received	0.015	(0.020)
Mid-day meal, etc. received	-0.017	(0.019)
Enrollment cost (Rs.)	0.000	(0.000)

Note:

Data from the 64th round of the NSS, “Participation and Expenditure in Education” in 2007–08. All standard errors are design-based, with PSUs clustered and nested within strata.

fig-district-maps-main (Figure 2)

```
knitr::include_graphics("collage1_map.png")
```

tab-iv-summary (Table 4)

```
kbl(
  joined_stats %>%
    select(Variable = label, Description = desc, Min, `1Q`, Med, `3Q`, Max, Mean, SD,
           N),
  booktabs=TRUE,
  longtable=TRUE, # longtable=T so table flows across pages
  label = "tab-iv-summary",
  caption="Summary Statistics for 2SLS Model",
  escape=FALSE, # escape=F so LaTeX labels can be used
  format = "latex"
) %>%
kable_styling(
  latex_options=c(
```

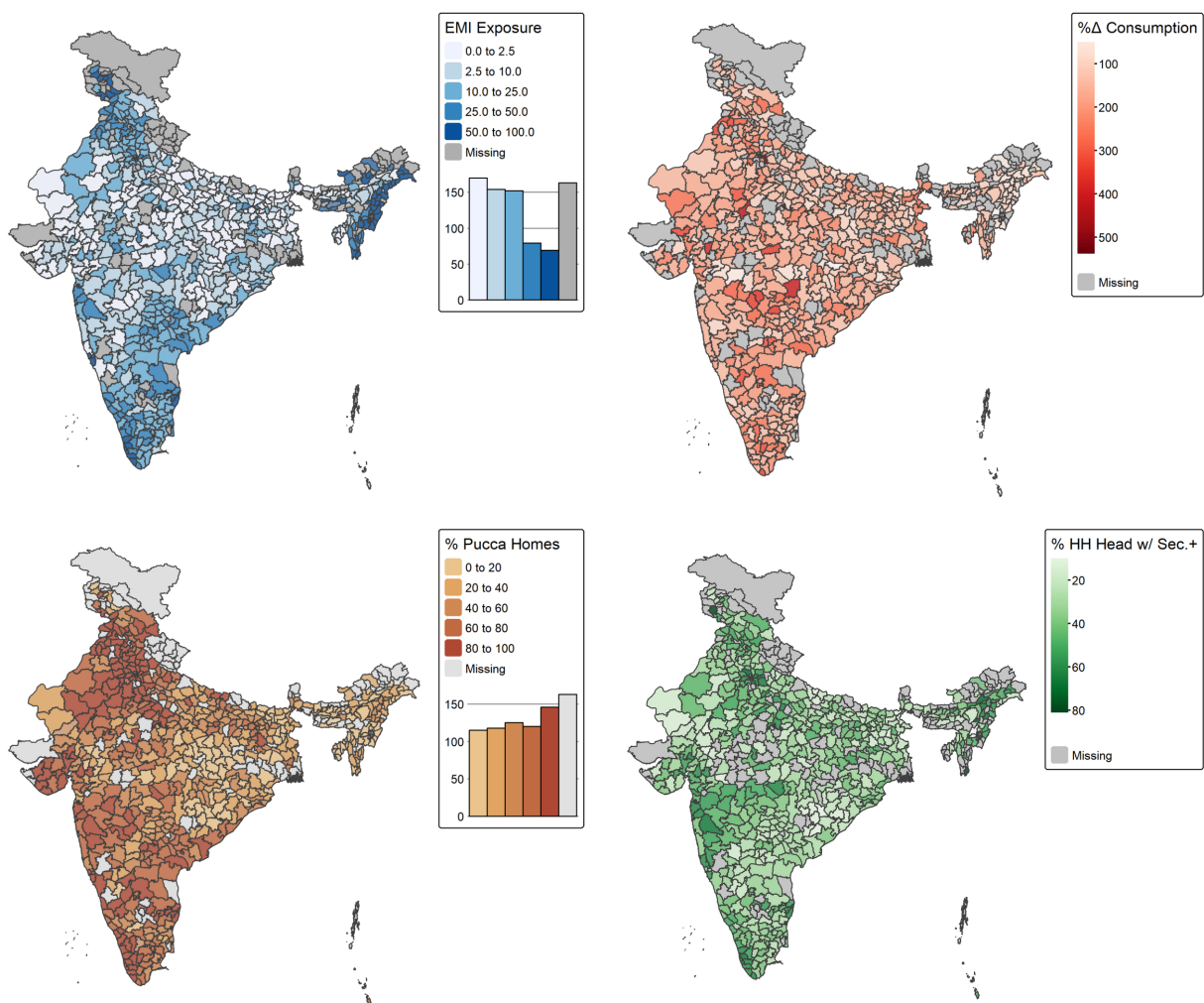


Figure 2: (Clockwise from top left) EMI exposure, consumption growth, pucca (permanent) housing, and household heads with secondary education or more. Data from the 64th round of the NSS 2007-08, “Participation and Expenditure in Education” and “Household Consumer Expenditure.”

```

    "repeat_header",
    "scale_down",
    "striped")
, full_width=FALSE
) %>% # Header repeats on each page, table scaled down so its full width fits on each
↪ page

## pack_rows, with row indices from joined_stats
pack_rows("From 2001:", 1, 1) %>%
pack_rows("From 2007-08:", 2, 22) %>%
pack_rows("From 2017-18:", 23, 25) %>%
pack_rows("From 2007-08 to 2017-18:", 26, 27) %>%
footnote(
  general = paste(
    "`Min.` = minimum;",
    "`1Q` = first quartile;",
    "`Med.` = median;",
    "`3Q` = third quartile;",
    "`Max.` = maximum;",
    "`Mean` = arithmetic mean;",
    "`SD` = standard deviation;",
    "`N` = number of observations",
    collapse = " "
  ),
  escape = FALSE,
  threeparttable = TRUE,
  footnote_as_chunk = TRUE
) %>%
landscape() %>%
# Narrow columns
column_spec(1, width = "4cm") %>% # Variable
column_spec(2, width = "6cm") # Description

```

Table 4: Summary Statistics for 2SLS Model

Variable	Description	Min	1Q	Med	3Q	Max	Mean	SD	N
From 2001:									
EMIE	EMI exposure	0.00	2.41	9.06	20.61	100.00	16.28	20.13	454
From 2007-08:									
Ling. Distance	Average linguistic distance of mother tongue from Hindi	0.00	0.06	1.77	4.17	5.00	2.14	1.93	454
Population	Estimated via NSS sample weights	31,492	1,056,344	1,662,361	2,654,010	9,922,640	1,979,019	1,389,793	454
Consumption	Average household monthly consumption expenditures (Rs.)	330.09	630.69	779.26	997.47	2923.14	861.53	339.82	454
Gini of Consumption	Gini coefficient of consumption	0.06	0.23	0.27	0.31	0.57	0.27	0.07	454
Pct. Urban	Percentage of people in an urban area	0.00	8.85	15.71	29.51	100.00	21.74	18.40	454
Avg. HH Size	Average household size	3.85	4.99	5.56	6.17	8.97	5.61	0.89	454
Dependency Ratio × 100	Ratio of dependents (0-14, 65+) to labor force (15-64), × 100	23.69	47.46	54.65	67.17	107.26	58.24	14.91	454
Pct. Female Head	Percentage of households with a female head	12.21	17.56	19.32	21.14	28.98	19.49	2.75	454
Pct. Hindu	Percentage of Hindus	0.00	77.61	89.17	95.24	100.00	80.66	22.73	454
Pct. Muslim	Percentage of Muslims	0.00	2.09	6.77	13.52	100.00	11.57	16.46	454
Pct. Other	Percentage not Hindu/Muslim	0.00	0.00	1.02	4.80	100.00	7.77	18.76	454
Pct. ST	Scheduled Tribe	0.00	0.00	2.02	11.05	100.00	11.79	21.43	454

Table 4: Summary Statistics for 2SLS Model (*continued*)

Variable	Description	Min	1Q	Med	3Q	Max	Mean	SD	N
Pct. SC	Scheduled Caste	0.00	11.53	18.37	25.82	48.86	18.75	10.23	454
Pct. OBC	Other Backward Class	0.00	22.18	44.93	60.73	88.32	42.10	22.76	454
Pct. Small Land-Owner	Owns 0.005–0.40 hectares	4.89	32.61	45.37	59.98	95.61	46.10	18.30	454
Pct. Med. Land-Owner	Owns 0.41–3.00 hectares	0.00	19.15	30.49	42.93	94.64	32.22	17.26	454
Pct. Large Land-Owner	Owns ≥ 3.01 hectares	0.00	0.00	1.78	6.18	43.73	4.25	6.16	454
Pct. Head Educ., Illiterate	Percentage of household heads with educ. level: illiterate	0.00	24.43	35.18	46.92	78.68	35.89	15.75	454
Pct. Head Educ., Lit.-Primary	Percentage of heads with educ. level: literate-primary	3.28	19.25	25.91	33.05	74.48	26.81	9.87	454
Pct. Head Educ., Secondary+	Percentage of heads with educ. level: above secondary	0.58	27.28	35.23	45.23	78.59	37.30	13.90	454
Pct. Pucca	Percentage in pucca (permanent) homes	0.00	32.68	58.79	81.64	100.00	56.85	27.89	454
From 2017-18:									
Population	Estimated via NSS sample weights	23,004	980,072	1,589,294	2,531,778	12,274,837	1,994,872	1,592,507	454
Consumption	Average household monthly consumption expenditures (Rs.)	771.50	1563.30	2077.24	2630.94	11031.46	2281.07	1030.43	454
Gini of Consumption	Gini coefficient of consumption	0.12	0.20	0.24	0.29	0.59	0.25	0.07	454

Table 4: Summary Statistics for 2SLS Model (*continued*)

Variable	Description	Min	1Q	Med	3Q	Max	Mean	SD	N
From 2007-08 to 2017-18:									
% Δ Consumption	Percent change in consumption	12.25	128.07	162.61	194.87	542.70	167.79	62.93	454
Δ Gini ^{Consumption}	Change in the Gini coefficient of consumption	-0.31	-0.07	-0.02	0.02	0.26	-0.02	0.07	454

Note: ‘Min.’ = minimum; ‘1Q’ = first quartile; ‘Med.’ = median; ‘3Q’ = third quartile; ‘Max.’ = maximum; ‘Mean’ = arithmetic mean; ‘SD’ = standard deviation; ‘N’ = number of observations

fig-regions-linguistic-distance (Figure 3)

```
knitr::include_graphics("collage2_map.png")
```

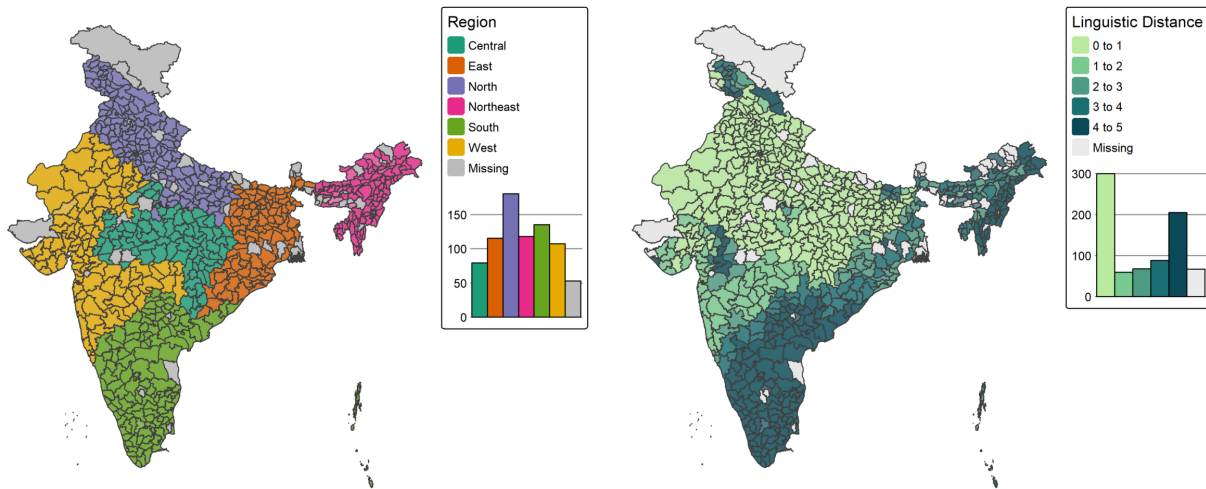


Figure 3: From left to right: regions of India and linguistic distance from Hindi. Data from the 2001 Census of India.

tab-first-stage-consumption (Table 5)

```
# Mapping for covariate labels
var_labels <- c(
  "EMIE" = "EMI exposure (fitted)",
  "wavg_ling_degrees" = "Linguistic distance",
  "consumption_0708" = "Consumption (2007-08)",
  "gini_cons_0708" = "Gini of Consumption (2007-08)",
  "avg_IMR" = "Average IMR",
  "pct_urban" = "Pct. Urban (ref: Rural)",
  "avg_hh_size" = "Average HH size",
  "dependency_ratio" = "Dependency ratio × 100",
  "pct_fem_head" = "Pct. Female head",
  "pct_hindu" = "Pct. Hindu (ref: Other)",
  "pct_muslim" = "Pct. Muslim",
  "pct_st" = "Pct. Scheduled Tribe (ref: Other)",
```

```

    "pct_sc" = "Pct. Scheduled Caste",
    "pct_obc" = "Pct. OBC",
    "pct_small_land" = "Pct. Small Land-owner (ref: No Land)",
    "pct_medium_land" = "Pct. Medium Land-owner",
    "pct_large_land" = "Pct. Large Land-owner",
    "pct_head_lit_to_primary" = "Pct. Head Educ., Lit.-Primary (ref: Illiterate)",
    "pct_head_secondary_plus" = "Pct. Head Educ., Secondary+",
    "(Intercept)" = "Intercept"
)

#### Test instrument's strength ####

# Run cluster-robust Wald F-test
vcov_partialf <- vcovCL(first_stage_consumption, cluster = joined_df$state_20)

partial_F_test <- linearHypothesis(
  first_stage_consumption,
  "wavg_ling_degrees = 0",
  vcov. = vcov_partialf,
  test = "F"
) # Wald F-stat with robust vcov + one endogenous regressor "is equivalent to the
  ↪ Kleibergen-Paap statistic" (https://lrberge.github.io/fixest/reference/fitstat.html)

# Extract F-stat and p-val
partial_f <- partial_F_test[["F"]][2]
partial_p <- partial_F_test[["Pr(>F)"]][2]

# Create asterisks based on p-val
stars <- function(p) {
  if (p < 0.001) return("***")
  else if (p < 0.01) return("**")

```

```

else if (p < 0.05) return("*")
else return("")
}

# Add custom GOF row for the partial F-stat
custom_gof_row <- data.frame(
  term = "Instrument's F-Statistic",
  estimate = paste0(formatC(partial_f, digits = 2, format = "f"), stars(partial_p)),
  stringsAsFactors = FALSE
)

# Table with styling and stats
modelsummary(
  first_stage_consumption,
  coef_map = var_labels,
  vcov = vcov_first_stage_consumption,
  gof_map = list(
    list(row = "nobs", clean = "Observations", fmt = 0),
    list(row = "r.squared", clean = "$R^2$", fmt = 3),
    list(row = "adj.r.squared", clean = "Adjusted $R^2$", fmt = 3),
    list(row = "sigma", clean = "Residual Std. Error", fmt = 3),
    list(row = "statistic", clean = "Model's F$-Statistic", fmt = 2)
  ),
  stars = c('*' = .05, '**' = .01, '***' = .001),
  fmt = 3,
  title = "First-Stage Regression: EMI Exposure on Linguistic Distance",
  output = "kableExtra",
  escape = FALSE,
  notes = list(
    "Standard errors clustered by state in parentheses."
  ),

```

```

add_rows = custom_gof_row
) %>%
kable_styling(latex_options = c(
  "hold_position",
  "repeat_header",
  "striped",
  "longtable"),
position = "center",
full_width = FALSE
) %>%
add_header_above(c(" " = 1, "EMI Exposure" = 1))

```

tab-second-stage-consumption-iv (Table 6)

```

modelsummary(
  model_consumption_iv,
  coef_map = var_labels,
  vcov = vcov_model_consumption_iv,
  gof_map = list(
    list(row = "nobs", clean = "Observations", fmt = 0),
    list(row = "r.squared", clean = "$R^2$", fmt = 3),
    list(row = "adj.r.squared", clean = "Adjusted $R^2$", fmt = 3),
    list(row = "sigma", clean = "Residual Std. Error", fmt = 3),
    list(row = "waldtest", clean = "F-Statistic", fmt = 2)
  ),
  stars = c('*' = .05, '**' = .01, '***' = .001),
  fmt = 3,
  title = "Second-Stage Regression: Consumption Growth on EMIE (Fitted)",
  output = "kableExtra",
  escape = FALSE,
  notes = list(
    "Standard errors clustered by state in parentheses."
  )
)

```

Table 5: First-Stage Regression: EMI Exposure on Linguistic Distance

	EMI Exposure
	(1)
Linguistic distance	2.945** (0.949)
Consumption (2007-08)	0.018*** (0.005)
Gini of Consumption (2007-08)	-40.452* (19.986)
Pct. Urban (ref: Rural)	0.245** (0.086)
Average HH size	-0.576 (2.059)
Dependency ratio $\times$ 100	-0.144 (0.089)
Pct. Female head	-0.415 (0.555)
Pct. Hindu (ref: Other)	-0.273* (0.109)
Pct. Muslim	-0.085 (0.183)
Pct. Scheduled Tribe (ref: Other)	0.015 (0.098)
Pct. Scheduled Caste	-0.012 (0.114)
Pct. OBC	-0.033 (0.088)
Pct. Small Land-owner (ref: No Land)	0.156* (0.078)
Pct. Medium Land-owner	0.167 (0.118)
Pct. Large Land-owner	0.115 (0.135)
Pct. Head Educ., Lit.-Primary (ref: Illiterate)	-0.319** (0.123)
Pct. Head Educ., Secondary+	0.193 (0.121)
Intercept	33.076 (27.314)
Observations	454
R^2	0.630
Adjusted R^2	0.615
Instrument's F-Statistic	9.63**

* $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$

Standard errors clustered by state in parentheses.

),

`add_rows = NULL`

```

) %>%
  kable_styling(
    latex_options = c("hold_position", "repeat_header", "striped", "longtable"),
    position = "center",
    full_width = FALSE
  ) %>%
  add_header_above(c(" " = 1, "Consumption Growth" = 1))

```

fig-district-carveouts-shifts (Figure 4)

```
plt_district_carveouts_shifts
```

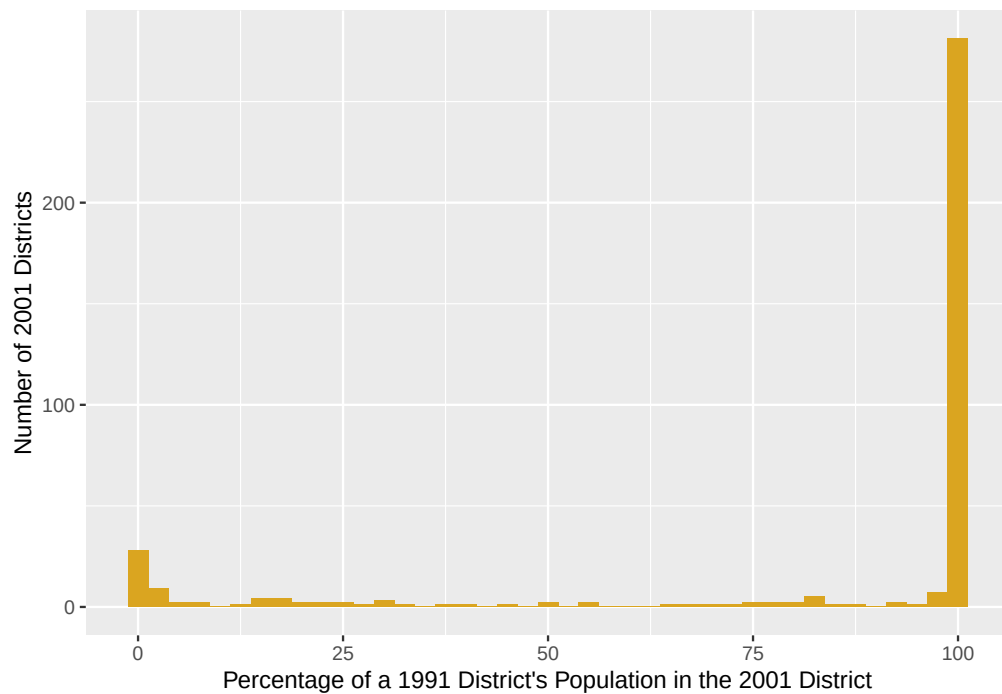


Figure 4: Number of 2001 districts which absorbed a percentage of a 1991 district's population via name change, clean merger, carve-out, or border shift. Data from Kumar & Somanathan (2016).

Table 6: Second-Stage Regression: Consumption Growth on EMIE (Fitted)

	Consumption Growth
	(1)
EMI exposure (fitted)	0.201 (0.710)
Consumption (2007-08)	-0.104*** (0.027)
Gini of Consumption (2007-08)	-11.654 (68.983)
Pct. Urban (ref: Rural)	1.240** (0.438)
Average HH size	-5.080 (6.090)
Dependency ratio $\times$ 100	-0.419 (0.263)
Pct. Female head	-1.807 (2.167)
Pct. Hindu (ref: Other)	-0.404 (0.232)
Pct. Muslim	-0.786** (0.277)
Pct. Scheduled Tribe (ref: Other)	-0.857*** (0.238)
Pct. Scheduled Caste	-0.223 (0.293)
Pct. OBC	-0.615*** (0.177)
Pct. Small Land-owner (ref: No Land)	-0.169 (0.313)
Pct. Medium Land-owner	-0.717 (0.459)
Pct. Large Land-owner	0.567 (0.395)
Pct. Head Educ., Lit.-Primary (ref: Illiterate)	-0.384 (0.466)
Pct. Head Educ., Secondary+	-0.494* (0.251)
Intercept	457.536*** (86.523)
Observations	454
R^2	0.232
Adjusted R^2	0.202

* $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$

Standard errors clustered by state in parentheses.